Aura: Reverse Engineer a Result

Project: APL-project 2026, Task 4 "Reverse Engineer a Result"
Live site: https://aura-recreate-aplkbr.lovable.app
Author: kabir-it

Project purpose

The goal of this task was to take a finished application, work out what code and logic produced it, and then rebuild it with the help of GenAI. The application I received was **Aura Enforcer (Alpha 0.1)**, a landing page for a Windows accessibility tool that finds and fixes digital barriers (low contrast, unlabelled icons, tiny touch targets) in real time. I only had screenshots of the finished result, so I had to work backwards, like a detective, from what I saw to the code behind it.

Phase 1: Research (Preliminary study)

Information gathering

- Studied the task sheet "Reverse Engineer a Result" (look at the application, ask questions, guess inputs, guess the process, sketch the steps, try building it).
- Studied the screenshots of the finished site: the layout, the colours, the texts, the buttons and every state of the page.
- Read the browser address bar in the screenshots (`.../aura/static/index.html`), which showed me that the original was a **static HTML page** and not a server-rendered application.

What I found in the screenshots

Observation	Conclusion
Dark navy background, blue/purple gradient on the word "Accessibility."	Custom CSS with CSS variables and `background-clip: text`
A dropdown with *The Voyager (Color Blind)*, *The*	Three accessibility "personas",

Beacon (Low Vision), *The Guardian (Eye Strain)*	each changing the colour theme
A toggle called *Clarify Content*	Swaps the page text to a simpler version ("The Guardian for everyone.")
A button that switches between *Simulate Aura Enforcer* and *Stop Enforcement*	A simulated mode that turns an overlay on and off
Yellow/blue outlines with labels like STRUCTURE, INTERACTION, Target Boosted, Readable, Optimized, Semantic	A visual overlay that marks which parts of the page the "enforcer" has fixed
A box: *Scanning: Notepad.exe, Fixes Applied: 12, Mode: ENFORCING*	A status panel with live-looking numbers
Sections "Universal Empowerment" with cards *Visual Enforcement*, *Aura Narrative*, *Vision Core*	A feature section with three cards
Footer "© 2026 Aura Systems. Part of the Tworam Ecosystem."	A simple footer

Guessed inputs and process (steps 3 and 4 of the task)

- **Inputs:** the persona the user picks, whether *Clarify Content* is on, and whether the enforcer is running.
- **Process:** the persona changes CSS variables, *Clarify Content* replaces text, and the enforcer button adds a CSS class to the page that draws the labelled outlines and starts a timer that updates the "Fixes Applied" number.

Choice of tools and technologies

- **HTML, CSS and vanilla JavaScript** for the first rebuild, because the original was a single static page.
- **Claude (GenAI)** to analyse the screenshots and generate the first version of the code.
- **Lovable** to rebuild and host the page online, since it was recommended in the task sheet.
- **Git and GitHub** for version control and publishing.

Plan

1. Rebuild the page as one `index.html` file and compare it with the screenshots.
2. Add the interactive parts: persona switcher, Clarify Content and the enforcer simulation.

3. Move the page to Lovable and publish it.
4. Document the work and push it to GitHub.

Problems and solutions in this phase

- **Problem:** The screenshots were rotated sideways, which made them hard to read.
  Solution: Read them carefully text by text and used the one upright screenshot as the main reference for the layout.
- **Problem:** The screenshots showed the page in several different states (different headline, different overlay colours), so it was unclear what was part of the page and what was added by something else.
  Solution: Compared the states one by one. I concluded that the yellow/blue labels were an overlay added on top of the page by the Aura tool itself, so I treated them as a separate "simulated enforcer" feature.
- **Problem:** Some text was cut off in the screenshots, so I could not read the original wording of all texts.
  Solution: Wrote the missing text myself in the same tone and marked it as my own wording.

Phase 2: Start of work (Implementation)

How the work started

I started by describing every visible part of the page and then asking GenAI to produce one self-contained `index.html` from that description. I chose a single file first so that I could test everything quickly in the browser before moving to Lovable.

Structure of the project

```
aura/
  static/
    index.html    <- the whole page (HTML + CSS + JavaScript)
README.md         <- this documentation
```

What was built first

1. **Page layout and theme.** CSS variables (`--bg` , `--accent` , `--glow` and so on) for colours so a theme can be changed in one place.
2. **Navigation bar.** The AURA logo on the left and the three controls on the right.

3. **Hero section.** The badge "Aura Enforcer | Alpha 0.1", the headline, the paragraph, the two buttons and the status box.
4. **Features section.** "Universal Empowerment" with three cards.
5. **Footer.**

How the interactive functions were developed

- **Persona switcher.** The `<select>` sets `data-persona` on the `<body>`. CSS then overrides the variables for each persona: blue for the Voyager, yellow with larger text for the Beacon, and a warm, dim theme for the Guardian (to reduce eye strain).
- **Clarify Content.** A toggle button with `aria-pressed`. When it is on, JavaScript replaces the headline, paragraph and card texts from a small object with two versions of the copy.
- **Simulate Aura Enforcer.** A function `setRunning(on)` adds the class `enforcing` to the `<body>`. CSS uses the `data-label` and `data-tag` attributes with `::before` and `::after` to draw the label chips (STRUCTURE, INTERACTION and so on) and the black tags (Readable, Target Boosted and so on). A timer updates "Fixes Applied" and the scanned app name every 1.8 seconds.

Example of the logic (step 5 of the task, a sketch of the function)

```
function setRunning(on) {
  body.classList.toggle('enforcing', on);        // show or hide the overlay
  run.textContent = on ? 'Stop Enforcement' : 'Simulate Aura Enforcer';
  modeEl.textContent = on ? 'ENFORCING' : 'IDLE';
  clearInterval(timer);
  if (on) timer = setInterval(updateFixesAndApp, 1800);
}
```

Problems and solutions in this phase

- **Problem:** Gradient text, labels and outlines clipped or overlapped neighbouring elements.
  Solution: Added padding around the elements and adjusted the label offsets until the overlay looked like the screenshots.
- **Problem:** The code started to become messy because the three themes and the overlay all touched the same elements.
  Solution: Moved all colours into CSS variables, kept the overlay in its own CSS section and kept all text versions in one JavaScript object.
- **Problem:** The custom dropdown looked wrong, with the white option list unreadable on

a dark theme.
Solution: Gave the `<option>` elements a light background with dark text.

- **Problem:** The overlay labels could not be placed on a `<select>` because pseudo-elements do not render on it.
  Solution: Wrapped each control in a `<div class="ctl">` and put the label attributes on the wrapper.

Phase 3: Finishing the work (Completion and improvement)

Completion

- Finished the page with all three personas, Clarify Content and the enforcer simulation.
- Moved the work to **Lovable** and published it at https://aura-recreate-aplkbr.lovable.app.

Testing the whole solution

- Switched between all three personas and checked that colours and text sizes changed.
- Turned *Clarify Content* on and off and checked that all texts changed back correctly.
- Started and stopped the enforcer and checked that the overlay, the button text and the status box all changed.
- Checked the page at a narrow width to see that the hero turns into one column.

Improvements and optimisation

- Moved repeated colours into variables.
- Added `aria-label` on the dropdown and `aria-pressed` on the toggle, which fits a project about accessibility.
- Made the status box fade when the enforcer is stopped so it is clear that it is idle.

Publishing

- The page is live on Lovable.
- The documentation and code are published on GitHub (see below).

Problems and solutions in this phase

- **Problem:** Lovable builds React projects, so a plain `index.html` cannot simply be uploaded there.
  Solution: Gave Lovable the HTML and asked it to rebuild the same page as a React project, then compared the result with my version.

- **Problem:** Things that worked in my local file could differ in the published version. **Solution:** Tested the published link again in the real environment and fixed small differences.

Phase 4: Problems and solutions (Summary)

Problem	Solution
Rotated and partly cut-off screenshots	Read them carefully, used the upright screenshot as the main layout reference and wrote missing texts in the same tone
Unclear what was the page and what was the enforcer overlay	Compared the screenshot states and treated the overlay as a separate simulated feature
Code and styles became messy with three themes	CSS variables, separated overlay CSS, one object for all text versions
Labels could not be drawn on a `<select>`	Wrapped controls in a `div` and put the label there
Lovable does not host a plain HTML file	Recreated the page as a React project from the HTML
Local version versus published version	Tested the live link and adjusted

---

Conclusion

What I achieved

I recreated the Aura Enforcer landing page from screenshots only, including its personas, the Clarify Content toggle and a simulated enforcer overlay, and published it online at https://aura-recreate-aplkbr.lovable.app. The code and this documentation are published on GitHub.

What I learned

- How to work backwards from a result: observe, guess the inputs, guess the process and

then build it.

- How CSS variables and data attributes can drive themes and overlays with very little JavaScript.
- How to use GenAI tools such as Claude and Lovable effectively: give clear, detailed descriptions and then check and correct the result myself.
- That an accessibility-focused product should itself follow accessibility practices (labels, contrast, large targets).

What could be improved

- The real Aura tool most likely works on actual Windows applications. My version only **simulates** this with a timer and fake numbers, so a real backend (for example a Python function that scans a window and returns the fixes) could be added.
- Some texts are my own wording because they were unreadable in the screenshots.
- More thorough testing on real phones and with real screen readers.
- Add automatic checks for colour contrast between the themes.